

Business-Scoped Customer Access

A multi-location foundation for ClientFlow CRM where every employee can assist the same customers without exposing data between unrelated businesses.

Proposed for team review

Local prototype

Working customer-list demo

Prepared for

ClientFlow CRM project team

Prototype branch

learning/company-customers

Scope

Tenant model, seed data, scoped query, read-only UI

Status

Not pushed or merged; decision requested

The proposal in one sentence

Treat each subscribing **Business** as the security boundary, let it own multiple **Locations**, and share its **Customers** with every active employee in that business.

1

Business tenant
The ownership and authorization boundary.

N

Locations
Physical branches under the same business.

1

Shared customer list
Available to active employees across locations.

Business rule

An employee of Business A can see Business A’s customers, regardless of location. The same employee cannot discover or access Business B’s customers. The server determines the business from the signed-in user; the browser does not choose a trusted business ID.

Why this fits the product

- Salons, stores, grooming businesses, and clinics commonly operate more than one location.
- A returning customer should not need a separate profile at every branch of the same business.
- Front-desk and service employees need the same customer context to provide consistent assistance.
- Unrelated businesses must remain strictly isolated even when their customers share an email or phone number.

Current prototype status

Area	Status	Evidence
Architecture decision	Proposed	Draft ADR 0012 on the local branch
Database structure	Implemented locally	Append-only migration 0002 applied to local PostgreSQL
Demo data	Working	Seed succeeds repeatedly without duplicate rows
Scoped read	Working	Query derives customers through user → employee → business
Frontend	Working prototype	<code>/customers</code> displays three seeded customers

What the existing design could not express safely

Before

- `Company_or_Location` mixed a tenant with a physical branch.
- Customers had no complete ownership path to the signed-in user.
- `Customer_Business` lacked a business foreign key.
- Customer email, phone, and address were globally unique.
- Appointments could not identify their location.

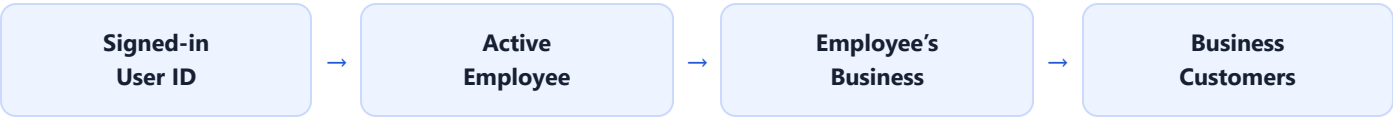
Proposed

- `Business` is the tenant and data boundary.
- `Location` belongs to a business.
- `Employee` links a user to a business.
- `Customer` belongs directly to one business.
- `Appointment` identifies a location.

Proposed relationship map



Authorization path



The customer query never accepts a client-provided business identifier. This follows the repository rule that every query is scoped by the current user.

Uniqueness becomes tenant-aware

Field	Old behavior	Proposed behavior
Customer email	Unique across the entire platform	Unique within one business
Customer phone	Unique across the entire platform	Unique within one business
Customer address	Globally unique	Not unique; households may share it
Location name/address	No separate location	Unique within its business

What has been implemented locally

File / area	Change	Purpose
schema.prisma	Business and Location models; tenant foreign keys; appointment location	Express ownership and multi-location operation
0002_business_tenancy.sql	Append-only SQL migration	Apply the model without altering migration 0001
seed.ts	Repeatable upserts for business, two locations, employee, and three customers	Provide a deterministic demo
package.json	Added <code>pnpm db:seed</code>	Expose the documented seed command
packages/domain	Added <code>listCustomersForUser(userId)</code>	Centralize the secure customer query
apps/web/app/customers	List page plus loading and error states	Show the database result through the UI
ADR-0012	Proposed tenant-boundary decision	Give the team a durable decision record to review

Demo dataset

Happy Paws Pet Care

- Downtown location
- North Side location
- One active demo employee

Shared customers

- Aisha Brown
- Maria Lopez
- James Wilson

What the demo proves

- ✓ The new migration applies to local PostgreSQL.
- ✓ The seed can run twice without creating duplicates.
- ✓ Customer results are ordered and scoped through employee membership.
- ✓ The Next.js page reads the records through the domain layer.
- ✓ Loading, empty, and recoverable error UI paths are represented.

Prototype boundary: this is a read-only customer-list slice. Customer creation, full access-control integration tests, real authentication, editing, deletion, search, pagination, and appointments remain follow-up work.

Why adopt this direction

Customer experience

A customer can visit another branch of the same business without recreating their profile or history.

Employee workflow

Any authorized employee can find the customer and continue service with consistent context.

Data isolation

Tenant-scoped queries prevent unrelated businesses from observing one another’s customer records.

Future growth

Locations become a clean foundation for appointment capacity, staff assignment, hours, and reporting.

Tradeoffs and open questions

Question	Current prototype choice	Team decision needed
Can one user work for multiple businesses?	No; User ↔ Employee is one-to-one	Use a membership model if multi-business users are required
Can employees work at multiple locations?	Not modeled yet	Add EmployeeLocation if scheduling requires it
Are services and roles tenant-owned?	Existing models remain global	Likely scope them to Business in a follow-up
What happens when records are removed?	No new delete behavior	Define soft-delete policy before CRUD deletion work
How is a business selected at login?	Dev user maps to one employee/business	Resolve during real authentication design

Alternatives considered

- **Location-owned customers:** rejected because customers must be shared across branches.
- **Global customer directory:** rejected because it creates privacy and tenant-isolation risks.
- **Many-to-many customers and businesses:** deferred because unrelated businesses should keep independent records, and the simpler ownership model is easier to secure.

Requested review and recommended next steps

The prototype is intended to start a team design conversation—not bypass the project’s issue, specification, review, and test process.

Please confirm

- 1 **Tenant boundary:** Business owns customer data; Location does not.
- 2 **Access policy:** all active employees in a business may read that business’s customers.
- 3 **Customer identity:** the same real person may have separate records at unrelated businesses.
- 4 **Near-term scope:** approve customer list/create before appointments and advanced scheduling.
- 5 **Future flexibility:** decide whether multi-business users and multi-location employees are required now or later.

If approved

1. Create the official feature issue.
2. Review and accept or revise ADR-0012.
3. Draft the customer list/create feature specification.
4. Reconcile the prototype with the reviewed documents.
5. Add tenant-isolation integration tests and create-customer validation.
6. Run `pnpm test`, `pnpm typecheck`, `pnpm build`, and the manual verification drill before a PR.

Recommendation: accept Business as the tenant boundary, then continue with the smallest complete vertical slice: list and create company customers with explicit cross-tenant rejection tests.